Congestion Control using ML/DL

Lab Project Report

Computer Networks (CN)

Slot F2, Fall Semester 2026–2027

Submitted by

Nirupam Raut (24BAI0345)

Aaditya Yadav (24BDS0450)

October 2026

Abstract

Congestion control decides how fast a TCP sender transmits. Rule-based algorithms such as Reno and CUBIC treat every packet loss as congestion and slow down only after the router queue has overflowed, so they lose most of their throughput on lossy links and keep queues full. This project asked whether machine learning (ML), deep learning (DL) and deep reinforcement learning (DRL) can remove these two weaknesses, and answered the question in eight stages, each prompted by the result of the one before.

A packet-level simulator with Reno, CUBIC and a BBR-style baseline came first. An XGBoost classifier, 91.4% accurate in telling congestion loss from random loss on unseen networks, was then placed inside CUBIC: at 5% random loss CUBIC+ML keeps 9.6 Mbps of a 10 Mbps link where CUBIC keeps 1.6, and it stays fair to CUBIC. An LSTM forecasts the round-trip time (RTT) 12% better than a naive forecast, and a Proximal Policy Optimization (PPO) agent uses 98.8% of capacity on unseen links at 1.07 times the minimum RTT. Comparing all six controllers exposed two weaknesses of the agent: it obtained 2% of a link shared with CUBIC, and it collapsed with 2 ms of RTT jitter. A competitive mode (PPO-Switch, 47% beside CUBIC) and training with random jitter (PPO-J, 99% utilisation at 4 ms) fixed both in simulation.

Finally, every controller was run over a real Wi-Fi network between two laptops. The classifier kept its benefit (8.7 against 2.9 Mbps under 1% loss) and so did the competitive mode (45% beside CUBIC), but the learned rate controllers did not, because the Wi-Fi stalls for 75 ms about twice a second. Trained on 29 minutes of recorded delay from that network, PPO-W raised the agent from 0.3 to 5.8 Mbps, as a replay of the recordings predicted. The project shows that a small learned component inside a proven algorithm is the most reliable gain, and that a learned controller needs real measurements, a reward that counts only what it controls, and tests on the real network before its simulated advantage can be trusted.

Keywords: congestion control, TCP, CUBIC, BBR, loss classification, XGBoost, LSTM, reinforcement learning, PPO, real-network test, trace-driven training.

Table of contents

Abstract 2
1. Introduction 6
1.1 Background 6
1.2 Problem statement 6
1.3 Aim and objectives 6
1.4 Scope and contributions 7
1.5 Organisation of the report 7
2. Literature review 9
2.1 Loss-based congestion control 9
2.2 Delay-based, model-based and network-assisted approaches 9
2.3 Throughput models and fairness measures 10
2.4 Distinguishing congestion loss from wireless loss 10
2.5 Learning-based congestion control 11
2.6 Machine-learning methods used in this project 12
2.7 Summary and research gap 12
3. Methodology 13
3.1 Overview of the approach 13
3.2 Users and use cases 14
3.3 Network simulator 14
3.4 Classical controllers 17
3.5 Machine learning: loss classifier 18
3.6 Loss-aware CUBIC (CUBIC+ML) 19
3.7 Deep learning: RTT forecaster 20
3.8 Deep reinforcement learning: PPO agent 20
3.9 Evaluation method 23
3.10 Tools and environment 24
4. Results and discussion 25
4.1 Stage 1: simulator and classical baselines 25
4.2 Stage 2: loss classifier and CUBIC+ML 26
4.3 Stage 3: RTT forecaster 27
4.4 Stage 4: training the PPO agent 28
4.5 Stage 5: comparison of all six controllers 28
4.6 Stage 6: fixing the weaknesses in simulation 33
4.7 Stage 7: test on a real Wi-Fi network 35
4.8 Stage 8: learning from the real network (PPO-W) 37
4.9 What each stage achieved 37
5. Conclusion 38
5.1 What we did and what we achieved 38
5.2 What we take from this project 38
5.3 Limitations and future work 38
References 39

List of figures

Figure 1: The eight stages of the project in the order they were carried out, with what each achieved 13
Figure 2: System architecture: one simulator produces the training data, trains the agents and evaluates every controller 14
Figure 3: Flowchart of one time step of the simulator 15
Figure 4: Class diagram: (a) the simulator and the classical controllers, (b) the learned controllers and the training environment 16
Figure 5: Sequence diagram of one decision interval of the PPO controller 17
Figure 6: Flowchart of the loss decision in CUBIC+ML 19
Figure 7: Congestion window (top) and round-trip time (bottom) of the three classical controllers, one flow each 25
Figure 8: Macro F1 score of the four loss classifiers on unseen scenarios 26
Figure 9: Mean absolute error of the LSTM and of three non-learned forecasts 27
Figure 10: Learning curves of the PPO agent and the hybrid: reward, share of capacity used and queueing delay 28
Figure 11: Link utilisation against delay on 30 unseen links; the best position is the top-left corner .. 29
Figure 12: Throughput against random loss rate for the classical (left) and learned (right) controllers 30
Figure 13: Throughput of each controller on a link whose capacity changes every 10 s 31
Figure 14: Utilisation (top) and delay ratio (bottom) under five conditions outside the training range 32
Figure 15: Share of a link beside one CUBIC flow (left) and Jain index between two flows of the same controller (right) 32
Figure 16: A CUBIC flow joins at 20 s and leaves at 40 s: PPO (left) and PPO-Switch, whose competitive mode is shaded (right) 34
Figure 17: RTT of every packet on the recorded Wi-Fi path: 60 s (top) and a 1.5 s close-up of two stalls (bottom) 36

List of tables

Table 1: Classical end-to-end congestion controllers 10
Table 2: Learning-based congestion controllers in the literature 11
Table 3: Parameters of the simulator 15
Table 4: The interface that every controller implements 16
Table 5: Ranges of the 400 scenarios used to generate training data 18
Table 6: The ten input features of the loss classifier 19
Table 7: Configuration of the LSTM forecaster 20
Table 8: The four signals that make up the agent's state 21
Table 9: Ranges of the links generated for each training episode 22
Table 10: PPO hyperparameters 22
Table 11: The ten trained policies 23
Table 12: Experiments used to compare the controllers 24
Table 13: Behaviour of the classical controllers on the standard link 25
Table 14: Performance of the four loss classifiers on 100 held-out scenarios (16,765 losses) 26
Table 15: Error of RTT forecasts 0.5 s ahead on 23,312 held-out windows 27
Table 16: Mean performance of the six controllers on 30 unseen links 29
Table 17: Throughput in Mbps on a 10 Mbps link under random loss (mean of 3 runs) 30
Table 18: Performance on the link with changing capacity 31
Table 19: Link utilisation (%) under conditions outside the training range 32

Table 20: Fairness between two flows on the standard link 33
Table 21: PPO-Switch compared with PPO, PPO-J and CUBIC+ML 34
Table 22: Utilisation (%) of one flow on the standard link under random RTT jitter (mean of 2 runs) 35
Table 23: Each controller alone: simulator, replay of the recorded Wi-Fi, and the real Wi-Fi path (second run) 36
Table 24: What each stage did, found and led to 37

1. Introduction

1.1 Background

The Transmission Control Protocol (TCP) carries most Internet traffic, and its congestion control algorithm decides how many packets a sender may have in flight at any time [1]. The decision matters because senders share links of limited capacity. If a sender transmits too slowly, capacity is wasted. If it transmits too fast, packets accumulate in the queue of the slowest router on the path (the bottleneck), delay rises, and the queue eventually overflows and drops packets. Congestion control was added to TCP after a series of congestion collapses in the 1980s, when Jacobson introduced slow start and congestion avoidance [2].

The sender has no direct view of the network. It learns about the path only from acknowledgements (ACKs): when they arrive, how long they took, and which packets were never acknowledged. Classical algorithms turn this feedback into a congestion window using fixed rules. The standard rule, additive increase and multiplicative decrease (AIMD), raises the window by one packet per round-trip time (RTT) and halves it when a packet is lost [3], [4]. CUBIC, the default in Linux, Windows and Apple operating systems, replaces the linear increase with a cubic function of time but keeps the same loss-driven logic [5], [6].

1.2 Problem statement

Loss-based congestion control has two well-known weaknesses.

The first is that every loss is treated as congestion. On wireless and other noisy links, packets are lost because of bit errors and interference while the queue is far from full. A loss-based sender cannot tell the difference and slows down. In the simulator built for this project, CUBIC falls from 10 Mbps to 3.7 Mbps when only 1% of packets are lost at random.

The second is that the sender reacts only after the queue overflows. Loss is a late signal, so the queue is kept close to full and every packet waits in it. In the baseline experiments of this project, Reno and CUBIC roughly double the RTT of a path: 79 to 88 ms on a path whose base RTT is 40 ms. With large buffers this behaviour causes the long queueing delays that motivated BBR [7].

A fixed rule also cannot adapt to links that differ by orders of magnitude in speed, delay and loss rate. This has motivated a large body of work on congestion control that is learned from experience instead of written by hand [8].

1.3 Aim and objectives

The aim of the project is to test whether learned components can remove the two weaknesses above, and to report clearly where they cannot. The objectives are:

1. To build a packet-level simulator of a bottleneck link and implement three classical baselines: Reno, CUBIC and a BBR-style controller.
2. Machine learning: to train a classifier that tells congestion loss from random loss using only what a sender can observe, and to use it inside CUBIC.
3. Deep learning: to train an LSTM network that forecasts a flow's round-trip time.
4. Deep reinforcement learning: to train a PPO agent that sets the sending rate directly, and a hybrid in which PPO adjusts CUBIC's window.
5. To compare all six controllers on throughput, delay, loss and fairness, including conditions not seen in training.
6. To fix the weaknesses that the comparison reveals, and to test the controllers on a real network.

1.4 Scope and contributions

Most of the work is carried out in simulation, with a final test over a real Wi-Fi network. Its contributions are:

- a packet-level simulator of a shared bottleneck link, written in Python, with a single interface through which every controller is driven;
- implementations of Reno, CUBIC and a simplified BBR, validated against the Mathis throughput model;
- a dataset of 64,256 loss events labelled with their true cause, and an XGBoost loss classifier with 91.4% accuracy on held-out scenarios;
- CUBIC+ML, a version of CUBIC that consults the classifier before reducing its window;
- an LSTM forecaster of RTT, evaluated against three non-learned forecasts;
- a PPO agent that controls the sending rate, a hybrid CUBIC+PPO controller, and six further trained policies used for ablation;
- PPO-Switch, which gives the agent a competitive mode for links shared with CUBIC, and PPO-J, an agent retrained to tolerate RTT jitter;
- a test bench that runs every controller over a real Wi-Fi network, recordings of that network, and PPO-W, an agent trained on them;
- a comparison of six controllers on unseen links, random loss, changing capacity, conditions outside the training range and fairness, and a live dashboard for demonstration.

1.5 Organisation of the report

Chapter 2 reviews classical and learning-based congestion control. Chapter 3 describes the simulator and the learned components, and shows the eight stages in which the work progressed. Chapter 4 presents the results stage by stage, each ending with what it established

and what it led to. Chapter 5 sets out what was achieved and what can be taken from the project, with its limitations and directions for future work.

2. Literature review

2.1 Loss-based congestion control

Jacobson's congestion avoidance and control algorithms [2] are the basis of TCP Tahoe, Reno and NewReno, and are standardised in RFC 5681 [4]. A connection starts in slow start, doubling its window every RTT, and then switches to congestion avoidance, in which the window grows by one packet per RTT. A loss halves the window. Chiu and Jain showed that this additive-increase, multiplicative-decrease rule is the linear control that converges to both efficiency and fairness [3].

AIMD is slow on fast long-distance paths, because recovering the window after a loss takes many round trips. CUBIC [5] addresses this by making the window a cubic function of the time since the last loss: growth is fast far from the previous maximum, slow near it, and independent of the RTT. CUBIC is now the most widely deployed congestion controller and is specified in RFC 9438 [6]. It remains loss-based, so it inherits both weaknesses described in Chapter 1.

2.2 Delay-based, model-based and network-assisted approaches

TCP Vegas [9] was the first widely studied delay-based scheme. It compares the expected and the actual sending rate, infers how many packets are queued, and adjusts the window to keep that number small. Vegas achieves low delay but obtains little throughput when it competes with loss-based flows, which keep filling the queue. Copa [10] targets a rate that is inversely proportional to the measured queueing delay; its authors note the same weakness and add a competitive mode that is activated when buffer-filling flows are detected.

BBR [7] takes a model-based approach. It estimates the bottleneck bandwidth from the maximum recent delivery rate and the propagation delay from the minimum recent RTT, and paces packets at the estimated bandwidth. Because it does not react to individual losses, BBR tolerates random loss; it can, however, be unfair to loss-based flows, and several BBR flows sharing a link build a standing queue.

A different line of work changes the network instead of the sender. Random Early Detection (RED) drops packets probabilistically before the queue is full, to signal congestion early [11]. DCTCP uses explicit congestion notification marks from switches and reduces the window in proportion to the fraction of marked packets [12]. These approaches need support from routers and are outside the scope of this project, which considers end-to-end control only. Table 1 summarises the end-to-end schemes.

Table 1: Classical end-to-end congestion controllers

Controller	Congestion signal	Behaviour	Known weakness
Reno / NewReno	Packet loss	Adds one packet per RTT; halves the window on loss	Slow on fast links; treats random loss as congestion
CUBIC	Packet loss	Cubic window growth; multiplies the window by 0.7 on loss	Keeps queues full; treats random loss as congestion
Vegas	Queueing delay	Keeps a small number of packets queued	Starved by loss-based flows
BBR	Delivery rate and minimum RTT	Paces at the estimated bottleneck rate	Can be unfair to loss-based flows; standing queue with several BBR flows
Copa	Queueing delay	Target rate inversely proportional to queueing delay	Low throughput beside buffer-filling flows unless it switches mode

2.3 Throughput models and fairness measures

Analytical models relate the throughput of a loss-based flow to the loss rate. Mathis et al. [13] showed that the steady-state throughput of Reno is

$$Throughput \approx (MSS / RTT) \times (1.22 / \sqrt{p}) \quad (1)$$

where MSS is the packet size and p is the loss probability. Padhye et al. [14] extended the model to include retransmission timeouts. These models explain why random loss is so damaging: a loss rate of 1% limits a Reno flow on a 40 ms path to a few megabits per second, whatever the capacity of the link. Equation (1) is used in Chapter 4 to validate the simulator. Fairness between flows is measured in this report with the index of Jain, Chiu and Hawe [15], which equals 1 when all flows receive equal throughput.

2.4 Distinguishing congestion loss from wireless loss

The idea of telling the two kinds of loss apart at the end host is not new. Biaz and Vaidya [16] classified losses at the receiver from packet inter-arrival times. TCP Westwood [17] estimates the available bandwidth from the ACK stream and, after a loss, sets the window from that estimate instead of halving it. TCP Veno [18] estimates the backlog at the bottleneck queue, as Vegas does, and treats a loss as random when the backlog is small, reducing the window by only one fifth. Each of these schemes relies on a hand-set threshold on one measured quantity. The classifier developed in this project uses ten sender-side signals and learns the decision boundary from labelled data.

2.5 Learning-based congestion control

Remy [19] was the first system to generate congestion-control rules by computer: given a model of the network and an objective, it searches offline for a rule table that maps observed state to actions. PCC [20] and its successor PCC Vivace [21] learn online instead: the sender tries slightly different rates, measures a utility built from throughput, loss and delay, and moves in the direction that improves it.

Reinforcement learning (RL) was applied by QTCP [22], which uses Q-learning to choose window adjustments, and by Indigo, a recurrent policy trained by imitation learning on the Pantheon test platform [23]. Aurora [24] formulated rate control as a deep RL problem. Its agent observes a history of latency gradient, latency ratio and sending ratio, and changes the sending rate multiplicatively; the same paper identifies fairness and generalisation as open problems. The agent in this project follows Aurora's formulation of state and action.

Later systems respond to those problems. Orca [25] keeps CUBIC running and lets a deep RL agent adjust its window at a coarser time scale, so that the classical scheme keeps the connection stable on unfamiliar links while the agent adapts it. Sage [26] learns offline from recorded behaviour of existing hand-designed schemes. Astraea [27] trains several agents together in a multi-flow environment so that fairness and convergence become part of the learned behaviour. A broad survey and experimental comparison of learning-based schemes is given by Jiang et al. [8]. Table 2 summarises the schemes discussed.

Table 2: Learning-based congestion controllers in the literature

Scheme	Year	Technique	Main idea
Remy	2013	Offline optimisation	Computer-generated rule table for a stated objective and network model
PCC / PCC Vivace	2015 / 2018	Online learning	Try rates, measure a utility, follow its gradient
QTCP	2019	Q-learning	Learn window adjustments online
Indigo (Pantheon)	2018	Imitation learning, LSTM	Recurrent policy trained on calibrated emulators
Aurora	2019	Deep RL (PPO)	Rate control from latency gradient, latency ratio and sending ratio
Orca	2020	Hybrid of CUBIC and deep RL	Agent corrects CUBIC's window periodically
Sage	2023	Offline (data-driven) RL	Learn from the behaviour of existing schemes
Astraea	2024	Multi-agent deep RL	Train competing agents together for fairness

2.6 Machine-learning methods used in this project

Three families of methods are used. For classification, gradient-boosted decision trees as implemented in XGBoost [28] and random forests [29] are compared with simpler models; both are ensembles of decision trees that perform well on small tabular datasets. For time-series forecasting, a long short-term memory (LSTM) network [30] is used; its gated recurrent cell retains information over many time steps. For control, the agent is trained with Proximal Policy Optimization (PPO) [31], a policy-gradient RL algorithm [32] that limits the size of each policy update and is known for stable training. The implementation uses the Stable-Baselines3 library [33], PyTorch [34] and scikit-learn [35].

2.7 Summary and research gap

The literature shows that loss-based control is simple and fair but fragile under random loss and slow to react; that delay-based and model-based control gives low delay but struggles to coexist with loss-based flows; and that learned controllers can outperform both on a single flow but raise questions of fairness, generalisation and safety. Most published systems are evaluated with kernel implementations and Internet testbeds that are beyond the reach of a course project. This project therefore reproduces the main ideas at small scale in one transparent simulator: a learned loss classifier inside an unmodified classical controller, a deep RL rate controller in the style of Aurora, and a hybrid in the style of Orca. Evaluating them side by side, with the same metrics and on the same links, shows what each idea contributes and where it fails.

3. Methodology

3.1 Overview of the approach

The work progressed in eight stages, shown in Figure 1. Stages 1 to 4 built the simulator and the three learned components; Stage 5 compared all six controllers and exposed two weaknesses of the agent; Stages 6 to 8 responded to what was found, first in simulation and then on a real Wi-Fi network. This chapter describes the components of Stages 1 to 4. Stages 6 to 8 were designed in response to results, so their methods are given with their results in Chapter 4. Figure 2 shows how the components are connected.

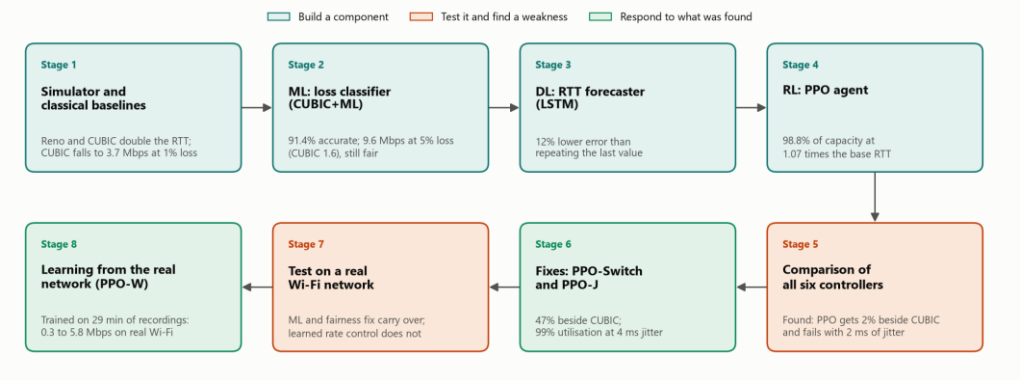


Figure 1: The eight stages of the project in the order they were carried out, with what each achieved

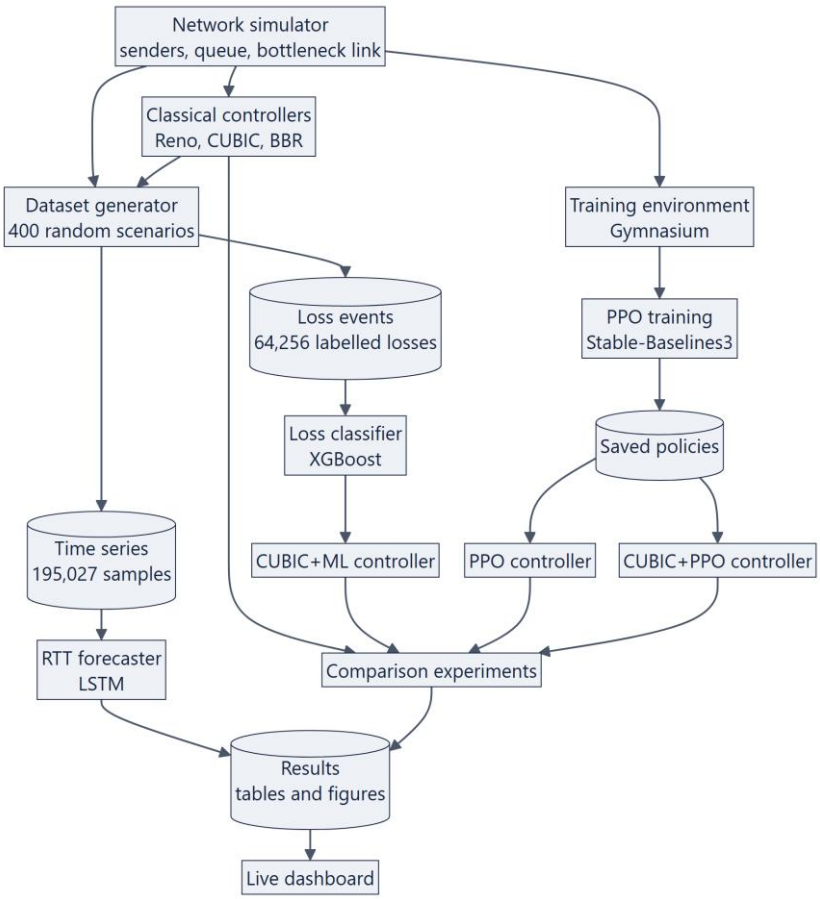


Figure 2: System architecture: one simulator produces the training data, trains the agents and evaluates every controller

Using one simulator for data generation, training and evaluation has two advantages. Every controller is tested on exactly the same network model, and the ground truth needed for training, namely the cause of each loss and the true link capacity, is always available.

3.2 Users and use cases

The system is a test bench for experiments. Its main user runs the baseline experiments, generates the training data, trains the classifier, the forecaster and the agents, compares all controllers, runs the real-network test and gives the live demonstration; an examiner or reader views the saved results and the demonstration.

3.3 Network simulator

3.3.1 Network model

The simulator models several senders that share one bottleneck. Packets from all senders enter a first-in-first-out queue of finite size in front of the bottleneck link. A packet that arrives when the queue is full is dropped (drop-tail). The link serves packets at its capacity, after which each packet's ACK returns to its sender over a path that is never congested. The link may also drop

packets at random with a fixed probability, which models non-congestion loss on a wireless link. Table 3 lists the parameters.

Table 3: Parameters of the simulator

Parameter	Meaning	Values used
Bandwidth	Capacity of the bottleneck; constant, or a function of time	0.5 to 100 Mbps
One-way delay	Propagation delay; the base RTT is twice this value	5 to 200 ms
Queue size	Capacity of the drop-tail queue	5 to 330 packets
Random loss rate	Probability that the link itself drops a packet	0 to 8%
Packet size	Fixed for all flows	1500 bytes
Time step	Resolution of the simulation	1 ms
Recording interval	Interval at which per-flow statistics are saved	100 ms

Time advances in steps of 1 ms. Figure 3 shows the work done in each step. Although the step is 1 ms, the departure time of every packet and the arrival time of every ACK are computed exactly within the step, so RTT and delivery-rate measurements are not rounded to whole milliseconds. Every lost packet is tagged with its true cause, congestion or random. Controllers cannot read this tag; it is used only to label training data and to score results.

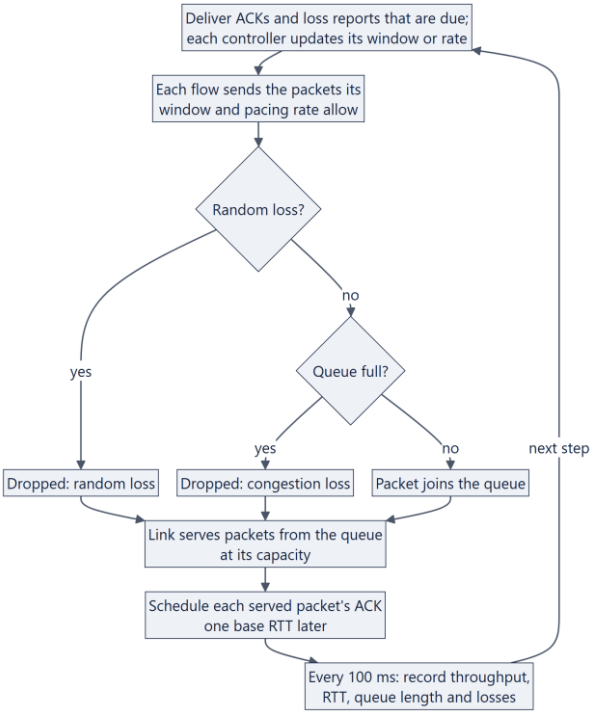


Figure 3: Flowchart of one time step of the simulator

3.3.2 Software design

Figure 4 shows the main classes. A Simulator owns one Link and one or more Flows, and each Flow owns a controller. Every controller, classical or learned, implements the same small

interface, listed in Table 4. This is what allows the six controllers to be compared on equal terms and to be mixed on one link.

Table 4: The interface that every controller implements

Member	Kind	Purpose
cwnd	Attribute	Congestion window, in packets
pacing_rate	Attribute	Sending rate in packets per second, or none for a purely window-based sender
on_ack(ack)	Callback	Called for every ACK, with the RTT, the delivery rate and the number of packets in flight
on_loss(loss)	Callback	Called for every reported loss
on_tick(now)	Optional callback	Called every time step; used by the RL controllers for their interval timer

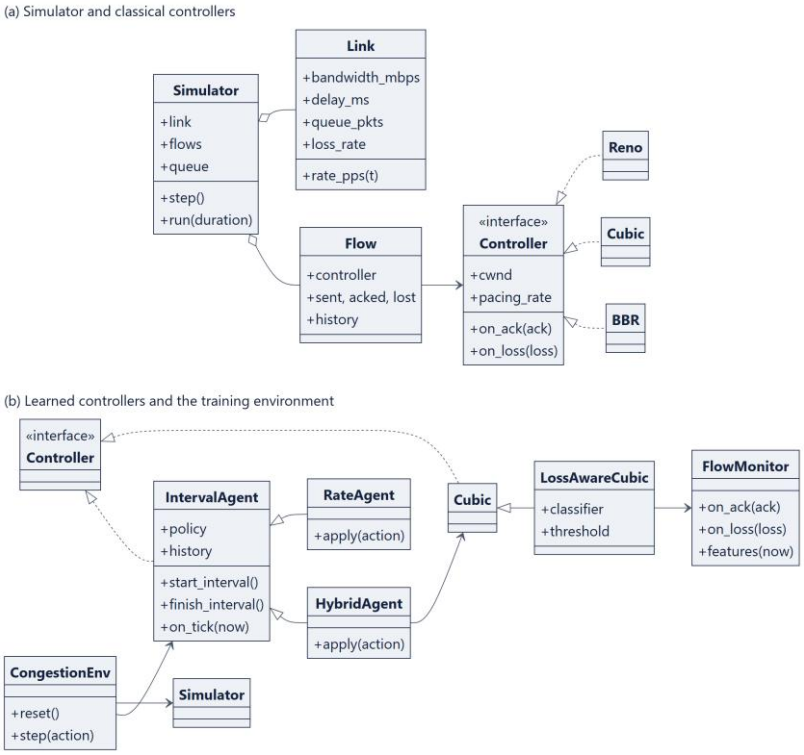


Figure 4: Class diagram: (a) the simulator and the classical controllers, (b) the learned controllers and the training environment

Figure 5 shows the messages exchanged during one decision interval of the PPO controller. The classical controllers take part in the same loop, but they update their window directly inside the on_ack and on_loss callbacks and have no policy network.

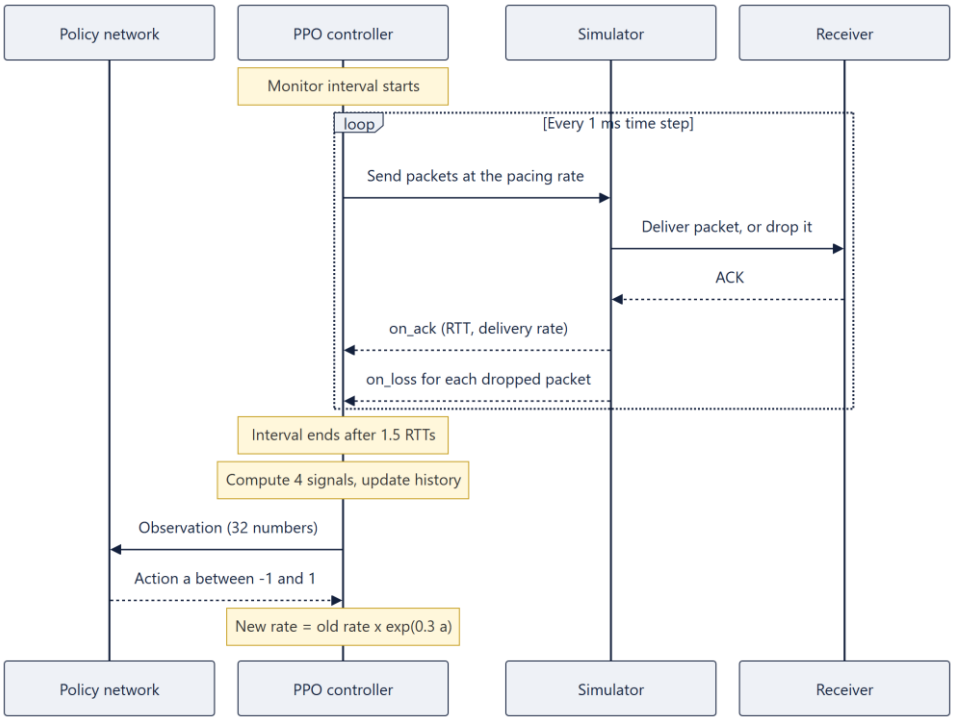


Figure 5: Sequence diagram of one decision interval of the PPO controller

3.3.3 Simplifications

There are no retransmission timeouts: every loss is reported to the sender about one RTT after it happens, as an ideal selective-acknowledgement receiver would. The ACK path is never congested, there is a single bottleneck, all flows have the same propagation delay, and the BBR baseline is a simplified version 1 of the algorithm.

3.4 Classical controllers

Reno follows RFC 5681. In slow start the window grows by one packet per ACK; in congestion avoidance it grows by 1/cwnd per ACK, that is, by one packet per RTT. When a loss is reported the window is halved. Further losses of packets that were sent before the reduction belong to the same congestion episode and are ignored.

CUBIC sets its window from the time t since the last reduction:

$$W(t) = C \cdot (t - K)^3 + W_{max}\,, \qquad K = (W_{max}\,(1 - \beta) / C)^{1/3} \tag{2}$$

where W_{max} is the window just before the last loss, $\beta = 0.7$ is the multiplicative decrease factor and $C = 0.4$. The implementation includes the TCP-friendly region and the fast-convergence rule of RFC 9438.

BBR is implemented as a simplified version 1. It keeps a windowed maximum of the delivery rate over ten round trips, which is its estimate of the bottleneck bandwidth, and a windowed minimum of the RTT over ten seconds. It paces at a gain times the estimated bandwidth, cycling the gain through 1.25, 0.75 and six phases of 1.0 to probe for more bandwidth and then drain the queue it created. Packets in flight are limited to twice the estimated bandwidth-delay

product. Every ten seconds it briefly reduces its window to four packets to measure the minimum RTT again.

3.5 Machine learning: loss classifier

3.5.1 Dataset generation

A real network does not reveal why a packet was lost, but the simulator does. Training data was therefore generated by running the three classical controllers in 400 randomly drawn scenarios of 30 simulated seconds each, with the ranges given in Table 5. Each time a controller was notified of a loss, the ten features of Section 3.5.2 and the true cause were saved. At most 200 events per class were kept from one scenario, so that fast, lossy links do not dominate.

Table 5: Ranges of the 400 scenarios used to generate training data

Setting	Range
Bandwidth	1 to 30 Mbps (log-uniform)
One-way delay	5 to 80 ms
Queue size	0.1 to 3 pipe sizes, at least 5 packets
Random loss	None in 15% of scenarios; otherwise 0.01% to 3% (log-uniform)
Changing capacity	In 40% of scenarios: a new level between 50% and 150% of nominal every 3 to 8 s
Flows	1 to 3 per scenario, each Reno, CUBIC or BBR; later flows join within the first 10 s
Duration	30 s of simulated time

Here the pipe size is the number of packets in flight that exactly fills the link with an empty queue, that is, the bandwidth multiplied by the base RTT. Small queues are deliberately included because they are the hard case: delay barely rises before a drop. The result is 64,256 labelled loss events, of which 49,092 are congestion losses and 15,164 are random losses. The same runs also produced 195,027 samples of per-flow measurements at 100 ms intervals, which are used for the forecaster of Section 3.7.

3.5.2 Features

Each loss is described by ten values that the sender can compute from its own ACKs and loss reports (Table 6). All are ratios, so they do not depend on the absolute speed or delay of the link, and a model trained on one set of links can be applied to others.

Table 6: The ten input features of the loss classifier

Feature	Meaning
rtt_ratio	Latest RTT divided by the minimum RTT seen
srtt_ratio	Smoothed RTT divided by the minimum RTT
rtt_peak_ratio	Latest RTT divided by the highest RTT seen recently; close to 1 when the queue is as full as it gets
rtt_trend	Change in RTT over the last 16 ACKs, in units of the minimum RTT
rtt_dev	RTT variation divided by the minimum RTT
rate_ratio	Delivery rate divided by the highest delivery rate seen recently
inflight_ratio	Packets in flight divided by the estimated pipe size
losses_last_rtt	Other losses reported within the last smoothed RTT
rtts_since_loss	Time since the previous loss, in smoothed RTTs
loss_rate	Loss rate over roughly the last 100 packets

3.5.3 Models and training

Four classifiers were trained on the same data: a single-threshold rule (a decision tree of depth one, which represents the hand-set thresholds of earlier schemes), logistic regression on standardised features, a random forest of 200 trees, and XGBoost with 300 trees of depth 6 and a learning rate of 0.1. Class weights compensate for the imbalance between the two classes.

The data was split by scenario, not by event: 297 scenarios (47,491 events) were used for training and 100 scenarios (16,765 events) were held out for testing; three of the 400 scenarios produced no loss. Splitting by scenario means that the test measures how well a classifier transfers to networks it has never seen, which is the situation it faces in use. The models are compared by accuracy, macro-averaged F1 score, per-class recall and the area under the ROC curve.

3.6 Loss-aware CUBIC (CUBIC+ML)

CUBIC+ML is standard CUBIC with one change, shown in Figure 6. A monitor object computes the ten features continuously from ACKs and loss reports, using the same code that generated the training data. When a loss is reported outside an ongoing recovery episode, the classifier estimates the probability that the loss was caused by congestion. If that probability is below a threshold the loss is ignored; otherwise CUBIC reduces its window as usual.

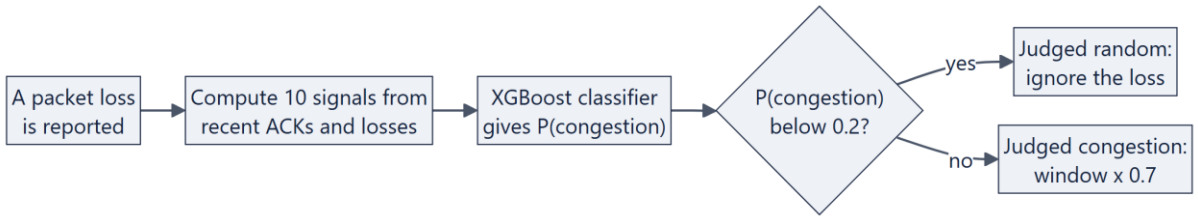


Figure 6: Flowchart of the loss decision in CUBIC+ML

The threshold controls a trade-off. A high threshold ignores more losses, which protects throughput under random loss but risks ignoring real congestion and taking bandwidth from competing flows. The threshold was set to 0.2: the window is reduced unless the classifier is at least 80% confident that the loss was random. Section 4.2.1 justifies this choice experimentally.

3.7 Deep learning: RTT forecaster

The forecaster predicts a flow's RTT 0.5 s ahead from the last 3 s of its own measurements. Its input is a sequence of 30 samples, one per 100 ms, of six signals: queueing delay relative to the minimum RTT, RTT relative to its running peak, throughput relative to its running peak, the fraction of packets lost, the time since the last loss, and the window size relative to its running peak. All are computed from past measurements only. The output is the change in RTT over the next 0.5 s, in units of the minimum RTT. Predicting the change instead of the value means that a model that outputs zero is identical to the naive forecast of repeating the last value, so any improvement over that forecast has been learned.

The network has one LSTM layer followed by a small dense head; Table 7 lists its configuration. The windows were split by scenario into 120,002 for training, 27,279 for validation and 23,312 for testing. Training stops when the validation loss has not improved for four epochs, and the best weights are restored. The LSTM is compared with three non-learned forecasts: the last value, a moving average over the last 0.5 s, and a linear extrapolation of the last 0.5 s.

Table 7: Configuration of the LSTM forecaster

Item	Value
Input	30 time steps of 6 signals (the last 3 s)
Recurrent layer	LSTM with 64 hidden units
Output head	Dense layer of 32 units with ReLU, then one output
Number of parameters	About 20,500
Loss function	Huber (smooth L1) loss
Optimiser	Adam, learning rate 0.001, batch size 512
Training length	At most 25 epochs; early stopping with a patience of 4 epochs
Forecast horizon	0.5 s (5 samples)

3.8 Deep reinforcement learning: PPO agent

3.8.1 State, action and reward

Congestion control is formulated as a sequential decision problem. The agent takes one decision per monitor interval, which lasts 1.5 smoothed RTTs and is bounded between 0.1 s and 1 s. During the interval it only gathers statistics; at the end it summarises them into the four signals of Table 8. The state given to the policy is these four signals for the last eight intervals,

32 numbers in total, so that the policy can see the effect of its recent actions. The first three signals follow Aurora [24]; the loss rate was added in this project.

Table 8: The four signals that make up the agent's state

Signal	Definition	What it indicates
Latency gradient	Change in RTT from the first to the second half of the interval, per second	Positive while a queue is building
Latency ratio	Mean RTT divided by the minimum RTT, minus 1	How much queueing delay exists
Send ratio	Packets sent divided by packets acknowledged, minus 1	Sending faster than the link delivers
Loss rate	Share of packets reported lost in the interval	Loss of either kind

The action is a single number a between −1 and 1 that scales the sending rate:

$$rate_{new} = rate_{old} \times e^{0.3a} \tag{3}$$

One decision can therefore raise the rate by up to 35% or lower it by up to 26%. A multiplicative change lets the same policy work on links of very different speed. The reward combines throughput, delay and loss:

$$r = T - w_d \cdot D - w_l \cdot L \tag{4}$$

where T is the delivered rate divided by the flow's fair share of the link capacity (capped at 1), D is the queueing delay divided by the base RTT (capped at 3), and L is the fraction of packets dropped by queue overflow. Random losses are not included in L, because the sender did not cause them. Every term is normalised by the link's own capacity or base RTT, so the reward has the same meaning on every link. The weights of the main agent are $w_d = 0.5$ and $w_l = 5$; two other weight profiles were also trained (Table 11). The reward uses the simulator's knowledge of the capacity and of the cause of each loss, which is available during training only. At test time the agent sees only its 32-number state.

3.8.2 Training environment

The simulator is wrapped as a Gymnasium environment. Each episode consists of 200 decisions on a newly generated random link (Table 9). Because every episode is a different network, the agent cannot memorise one link and has to learn a general rule.

Table 9: Ranges of the links generated for each training episode

Setting	Range
Bandwidth	2 to 40 Mbps (log-uniform)
One-way delay	5 to 75 ms (base RTT 10 to 150 ms)
Queue size	0.2 to 3 pipe sizes, at least 10 packets
Random loss	None in half of the episodes; otherwise 0.01% to 3%
Changing capacity	In half of the episodes: a new level between 50% and 150% of nominal every 3 to 8 s
Starting rate	Random, between 5% and 150% of capacity
Competing flow	Only for some policies: a Reno or CUBIC flow in 30% of episodes

3.8.3 Training procedure

The policy is a neural network with two hidden layers of 64 units, trained with PPO as implemented in Stable-Baselines3; Table 10 lists the hyperparameters. Eight copies of the environment run in parallel. After every 8,192 decisions the policy is updated, and this cycle is repeated about 120 times, for a total of 1,000,000 decisions (about 5,000 episodes). Training the main agent takes about 20 minutes on a laptop processor without a GPU.

Table 10: PPO hyperparameters

Hyperparameter	Value
Policy network	Two hidden layers of 64 units
Parallel environments	8
Decisions per environment per update	1,024 (8,192 in total)
Batch size and passes per update	256 and 10
Discount factor and GAE parameter	0.99 and 0.95
Learning rate	0.0003
Training length of the main agent	1,000,000 decisions

The controller class that collects the interval statistics is the same in training and at test time. In training the environment supplies the action; at test time the saved policy supplies it. The agent is therefore always evaluated on exactly the kind of observations it was trained on.

3.8.4 Hybrid controller and trained variants

The hybrid controller, CUBIC+PPO, follows the idea of Orca [25]. CUBIC runs on every ACK as usual. Once per monitor interval a PPO policy multiplies CUBIC's window by the same factor as in Equation (3), and CUBIC continues from the corrected window. The state and the reward are unchanged. In 30% of its training episodes a competing Reno or CUBIC flow shares the link. Six further policies were trained to answer specific questions (Table 11): whether the

reward weights steer the agent, whether the history and the loss-rate input are needed, and whether training with competing traffic improves fairness.

Table 11: The ten trained policies

Policy	Difference from the main agent	Decisions
PPO (main agent)	Sets the sending rate; balanced reward weights	1,000,000
CUBIC+PPO (hybrid)	Action scales CUBIC's window; competing flow in 30% of episodes	1,000,000
Competing traffic	A competing flow in 30% of episodes	1,000,000
Shorter training	Same as the main agent; reference for the variants below	500,000
Throughput profile	Reward weights $w_d = 0.1$ and $w_l = 2$	500,000
Low-latency profile	Reward weights $w_d = 2$ and $w_l = 5$	500,000
No history	State holds only the latest interval	500,000
No loss-rate input	Loss rate removed from the state	500,000
PPO-J (Section 4.6.2)	Random RTT jitter in 75% of episodes; latency signals from the minimum RTT	1,000,000
PPO-W (Section 4.8)	As PPO-J, plus replay of recorded Wi-Fi delay in half of the episodes; reward counts the bottleneck queue	1,000,000

3.9 Evaluation method

Table 12 lists the experiments. Where a single representative network is needed, a link of 10 Mbps with a base RTT of 40 ms and a queue of 50 packets is used.

Table 12: Experiments used to compare the controllers

Experiment	Setup
Unseen links	Each controller alone for 30 s on the same 30 random links, drawn from the training ranges with new random seeds; the first 5 s are excluded
Random loss	One flow on the standard link with random loss of 0, 0.1, 0.5, 1, 2 and 5%; mean of 3 runs
Changing capacity	One flow for 50 s on a link whose capacity steps through 10, 4, 14, 7 and 12 Mbps
Outside the training range	100 Mbps link; 400 ms base RTT; 8% random loss; 5-packet queue; 0.5 Mbps link
Fairness	Two flows on the standard link for 60 s, the second joining after 5 s, measured from 20 s; against CUBIC in both start orders, and against a copy of itself
Agent variants	Each trained policy on the 30 unseen links, and beside one CUBIC flow
Competitive mode	PPO-Switch beside one CUBIC, Reno or BBR flow in both start orders; a CUBIC flow that joins at 20 s and leaves at 40 s
RTT jitter	One flow on the standard link with random return-path jitter of mean 0, 1, 2, 4 and 8 ms; mean of 2 runs
Real Wi-Fi	Each controller alone, with 1% random loss, and beside CUBIC, over Wi-Fi between two laptops through an emulated 10 Mbps bottleneck; run twice

Four measures are reported. Link utilisation is the throughput as a percentage of the capacity available to the flow. The delay ratio is the average RTT divided by the base RTT, so that 1.0 means an empty queue. The loss rate is the percentage of sent packets that were lost; congestion loss counts only packets dropped by queue overflow. Fairness between n flows with throughputs x_i is measured by Jain's index [15]:

$$J = (\Sigma x_i)^2 / (n \cdot \Sigma x_i^2) \quad (5)$$

To compare agent variants with a single number, the balanced training objective is also evaluated over whole runs:

$$O = U - 0.5 \cdot \min(\max(R - 1, 0), 3) - 5 \cdot L_c \quad (6)$$

where U is the utilisation as a fraction, R is the delay ratio and L_c is the congestion-loss fraction. Because this is the quantity that the PPO agent was trained to maximise, it favours PPO by construction; the individual measures are therefore always reported alongside it.

3.10 Tools and environment

Everything is written in Python 3.12, with scikit-learn 1.9, XGBoost 3.4, PyTorch 2.14, Gymnasium 1.4, Stable-Baselines3 2.9, NumPy, pandas, Matplotlib, Streamlit and Plotly. It runs on a laptop with an Intel Core i5-1135G7 processor and no GPU; the real-network test adds a MacBook over Wi-Fi.

4. Results and discussion

This chapter follows the eight stages of the project in the order in which they were carried out; each stage ends with what it established and what it led to. All results come from the project's own simulator, models and real-network test. Unless stated otherwise, the link has a capacity of 10 Mbps, a base RTT of 40 ms and a queue of 50 packets.

4.1 Stage 1: simulator and classical baselines

Figure 7 shows one flow of each classical controller alone on the standard link. Reno and CUBIC produce the well-known sawtooth: the window grows until the queue overflows, is cut, and grows again, so the RTT swings between about 55 ms and 100 ms. BBR holds its window steady and keeps the RTT close to 40 ms, with a brief dip every 10 s when it measures the minimum RTT again. Table 13 summarises the measurements.

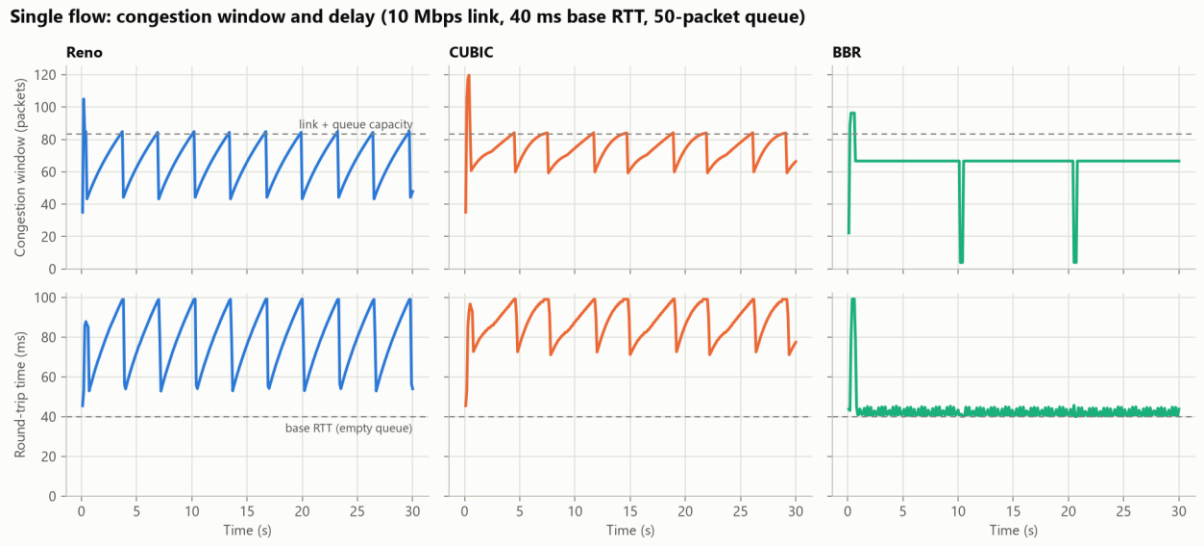


Figure 7: Congestion window (top) and round-trip time (bottom) of the three classical controllers, one flow each

Table 13: Behaviour of the classical controllers on the standard link

Controller	Throughput (Mbps)	Average RTT (ms)	95th percentile RTT (ms)	Jain index, two flows
Reno	10.00	79.1	98.5	1.000
CUBIC	10.00	88.1	99.0	0.995
BBR	9.82	41.9	44.8	0.999

The simulator was validated against the Mathis model of Equation (1). For an RTT of 40 ms and a loss probability of 1%, the model predicts 305 packets per second, or 3.7 Mbps. The simulated Reno flow achieves 3.9 Mbps. The agreement gives confidence that the loss and feedback mechanisms of the simulator behave correctly.

The baselines confirm both weaknesses stated in Chapter 1. Reno and CUBIC roughly double the RTT, because they keep the queue between half full and full. Under 1% random loss both fall below 4 Mbps; Section 4.5.2 gives the full sweep. Two BBR flows share the link fairly but build a standing queue that raises the RTT to 80 ms, a known behaviour of BBR version 1.

Stage 1 thus confirmed both weaknesses of loss-based control, in a simulator that agrees with theory. Stage 2 addresses random loss and Stage 4 the full queue.

4.2 Stage 2: loss classifier and CUBIC+ML

Table 14 and Figure 8 compare the four classifiers on the 100 held-out scenarios. The two tree ensembles reach a macro F1 score of 0.88, about three points above the single-threshold rule and logistic regression. A single threshold on one signal, as used by earlier loss-differentiation schemes, therefore leaves accuracy unused. XGBoost was selected for deployment: it is as accurate as the random forest and much faster when classifying one loss at a time, which is how a controller uses it.

Table 14: Performance of the four loss classifiers on 100 held-out scenarios (16,765 losses)

Model	Accuracy	Macro F1	Random losses recognised	Congestion losses recognised	ROC AUC
Single-threshold rule	0.891	0.852	78.1%	92.6%	0.853
Logistic regression	0.887	0.848	80.0%	91.4%	0.929
Random forest	0.915	0.883	81.3%	94.8%	0.946
XGBoost (deployed)	0.914	0.882	81.4%	94.6%	0.947

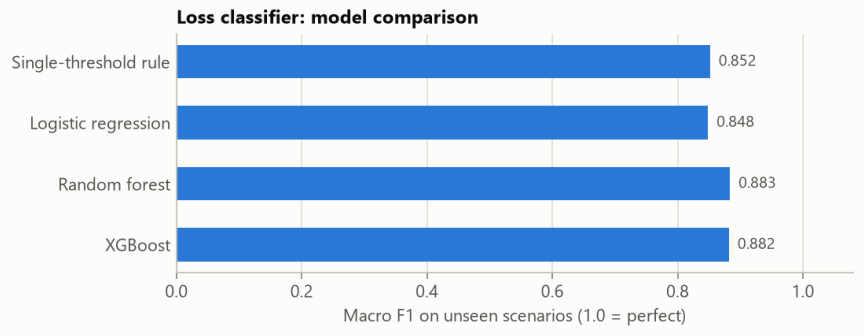


Figure 8: Macro F1 score of the four loss classifiers on unseen scenarios

The deployed model's errors are asymmetric in a useful way. It misses 5.4% of congestion losses, which is the dangerous error, and it misreads 18.6% of random losses as congestion, which only costs some throughput. Measured by the drop in macro F1 when a feature is shuffled, the classifier relies most on the RTT relative to its recent peak: a congestion loss happens when the queue is as full as it gets, whereas a random loss can occur at any queue level. The recent loss rate and the time since the previous loss follow. Accuracy stays between 0.89 and 0.93 across buffer sizes, including the shallow buffers that leave little delay signal before a drop.

4.2.1 Choosing the decision threshold

Throughput under random loss depends little on the decision threshold: even at 0.05 the controller keeps 9.5 Mbps at 5% loss. Fairness depends on it strongly. With a threshold of 0.5 the controller ignored some real congestion losses and took 73% of the link from a standard CUBIC flow that joined 5 s later. At 0.2 the two flows split the link evenly, and this value was adopted.

Stage 2 removed the random-loss weakness without making CUBIC unfair. The queue stays full, because the controller underneath is still loss-based; that is the target of Stages 3 and 4.

4.3 Stage 3: RTT forecaster

Table 15 and Figure 9 compare the LSTM with the three non-learned forecasts on held-out scenarios. The LSTM has the lowest error: its mean absolute error is 7.91 ms against 8.95 ms for repeating the last value, an improvement of 12%, and its root mean square error is 21% lower. The larger gain in the squared error indicates that the LSTM mainly avoids the largest misses. It improves on the last-value forecast for flows of all three controllers: from 8.71 to 6.97 ms for CUBIC, from 8.81 to 7.85 ms for BBR and from 9.29 to 8.81 ms for Reno.

Table 15: Error of RTT forecasts 0.5 s ahead on 23,312 held-out windows

Forecast method	Mean absolute error (ms)	Root mean square error (ms)	Mean absolute error (% of minimum RTT)
Last value	8.95	19.14	10.5
Moving average over 0.5 s	9.06	20.33	10.4
Linear extrapolation	15.69	32.50	18.5
LSTM	7.91	15.13	9.4

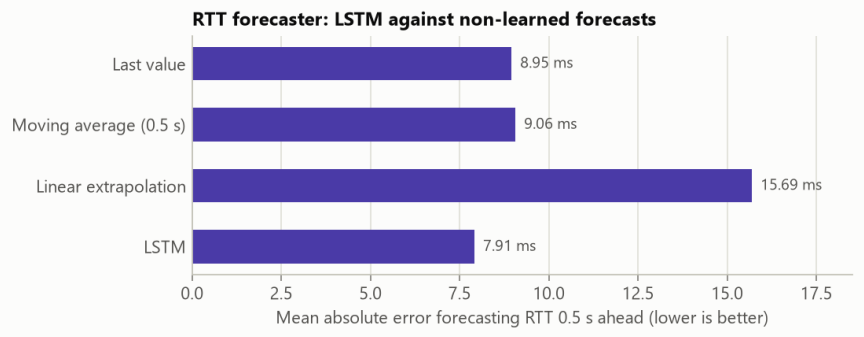


Figure 9: Mean absolute error of the LSTM and of three non-learned forecasts

The gain is modest. The LSTM anticipates some of the recoveries but, like the naive forecast, it lags sudden jumps, which cannot be predicted from a flow's own history when they are caused by events outside the flow.

Stage 3 showed that a flow's RTT is only partly predictable from its own history. The forecaster was therefore kept as a stand-alone result, and Stage 4 lets an agent learn to control the queue directly.

4.4 Stage 4: training the PPO agent

Figure 10 shows the progress of training for the PPO agent and for the hybrid. The PPO agent passes 90% link utilisation after about 200,000 decisions and continues to lower its queueing delay for the rest of training, reaching a mean reward of about 0.92 out of a maximum of 1. The hybrid's curve is lower and noisier. Part of the difference is expected, because 30% of its episodes contain a competing flow, in which queueing delay cannot be avoided.

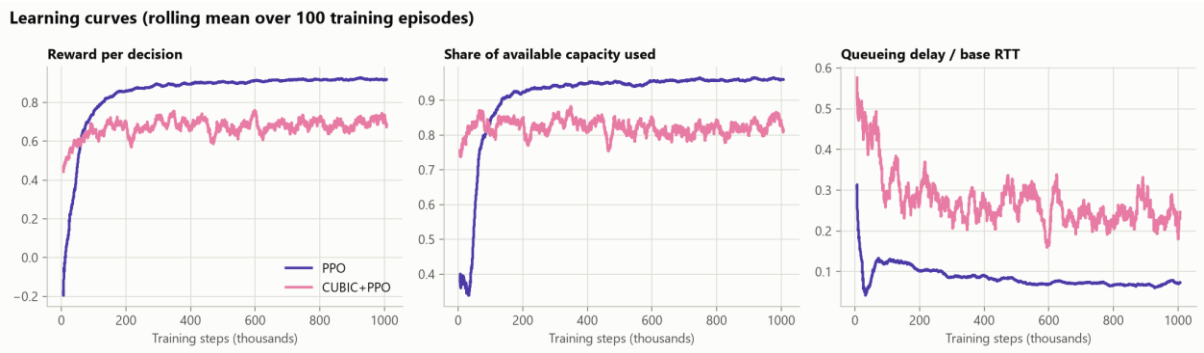


Figure 10: Learning curves of the PPO agent and the hybrid: reward, share of capacity used and queueing delay

The trained agent, the hybrid and the four other controllers are compared in Stage 5.

4.5 Stage 5: comparison of all six controllers

Stage 5 tests all six controllers under the same conditions: unseen links, random loss, changing capacity, conditions outside the training range, and links shared by two flows.

4.5.1 Unseen links

Each controller was run alone for 30 s on the same 30 links, drawn from the training ranges with new random seeds; half of them have random loss and half have changing capacity. Figure 11 plots the mean utilisation against the mean delay ratio, and Table 16 gives all measures.

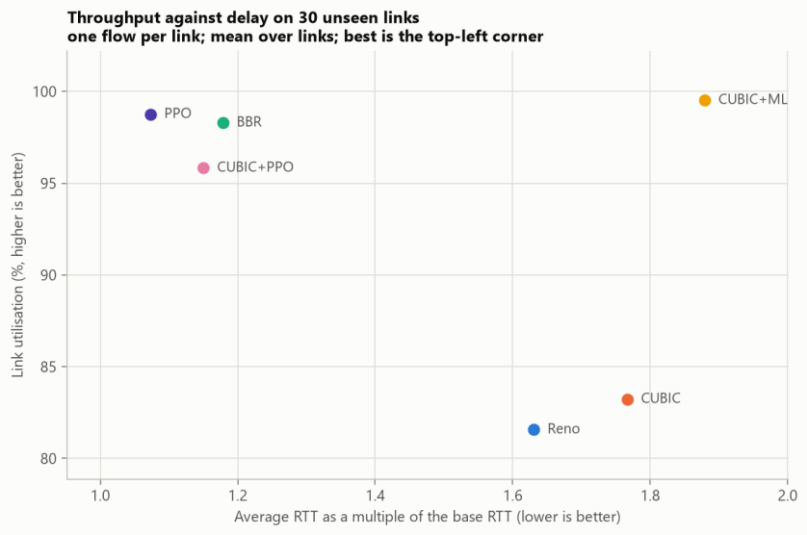


Figure 11: Link utilisation against delay on 30 unseen links; the best position is the top-left corner

Table 16: Mean performance of the six controllers on 30 unseen links

Controller	Utilisation (%)	RTT / base RTT	95th pct RTT / base RTT	Congestion loss (%)	Objective
Reno	81.6	1.63	2.13	0.15	0.493
CUBIC	83.2	1.77	2.18	0.15	0.441
BBR	98.3	1.18	1.65	1.11	0.838
CUBIC+ML	99.5	1.88	2.32	0.30	0.541
PPO	98.8	1.07	1.25	0.20	0.941
CUBIC+PPO	95.8	1.15	1.25	0.06	0.880

PPO uses 98.8% of capacity with an RTT only 7% above the minimum. It matches BBR's throughput with less delay and about a fifth of BBR's congestion loss. CUBIC+ML lifts CUBIC's utilisation from 83.2% to 99.5%, but it is still a loss-based controller and still fills the queue, with a delay ratio of 1.88. CUBIC+PPO lies between the two, at 95.8% utilisation with low delay and the lowest congestion loss. Reno and CUBIC lose throughput on the lossy links and still nearly double the RTT.

4.5.2 Random loss

Figure 12 and Table 17 show the throughput of one flow as random loss increases. CUBIC+ML holds at least 9.97 Mbps up to 2% loss and 9.6 Mbps at 5%, where plain CUBIC delivers 1.6 Mbps, a sixfold difference. The PPO agent does as well, although nothing tells it which losses are random: it has learned that loss without rising delay is no reason to slow down. CUBIC+PPO improves on CUBIC (8.4 against 3.7 Mbps at 1% loss) but degrades at higher loss, because the CUBIC part still cuts its window at every loss and the agent's corrections only partly make up for it.

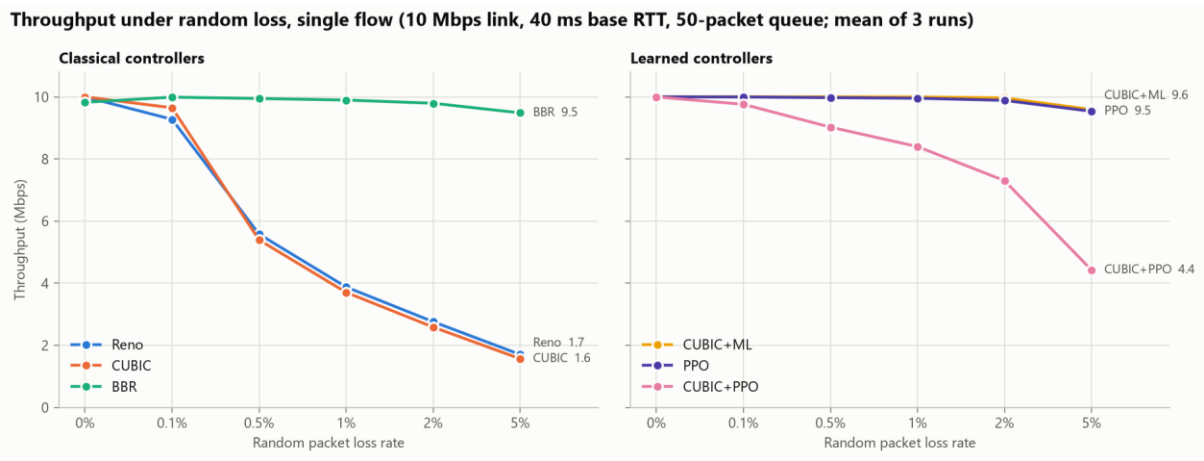


Figure 12: Throughput against random loss rate for the classical (left) and learned (right) controllers

Table 17: Throughput in Mbps on a 10 Mbps link under random loss (mean of 3 runs)

Controller	0%	0.1%	0.5%	1%	2%	5%
Reno	10.00	9.28	5.58	3.88	2.76	1.70
CUBIC	10.00	9.65	5.39	3.71	2.59	1.57
BBR	9.83	9.99	9.95	9.90	9.79	9.48
CUBIC+ML	10.00	10.00	10.00	10.00	9.97	9.59
PPO	10.00	10.00	9.98	9.96	9.89	9.54
CUBIC+PPO	10.00	9.76	9.03	8.40	7.30	4.43

4.5.3 Changing capacity

Figure 13 shows each controller on a link whose capacity changes every 10 s, and Table 18 gives the measurements. All six controllers use at least 97% of the available capacity; they differ in delay. When the capacity drops, Reno, CUBIC and BBR are left with a full queue for a while: the 95th percentile RTT reaches 4.4 to 4.6 times the base for Reno and CUBIC and 3.1 times for BBR. For PPO and CUBIC+PPO it stays at 1.2 times the base. The cost for PPO is a slower start: it needs about 1.5 s to climb from its initial rate of 1 Mbps.

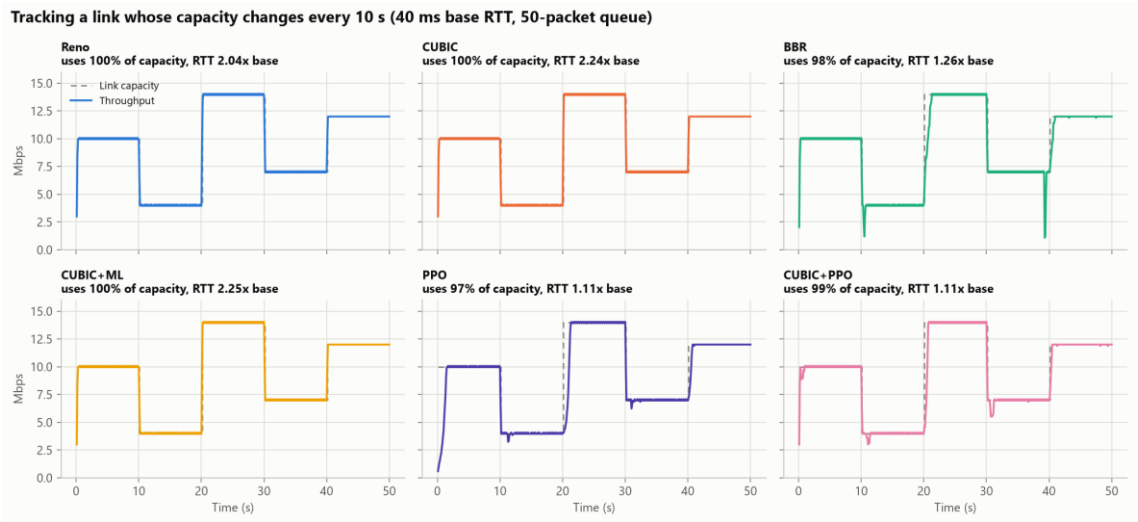


Figure 13: Throughput of each controller on a link whose capacity changes every 10 s

Table 18: Performance on the link with changing capacity

Controller	Utilisation (%)	RTT / base RTT	95th pct RTT / base RTT	Congestion loss (%)
Reno	99.96	2.04	4.38	0.09
CUBIC	99.98	2.24	4.55	0.08
BBR	97.96	1.26	3.12	0.69
CUBIC+ML	99.98	2.25	4.55	0.30
PPO	97.46	1.11	1.22	0.29
CUBIC+PPO	98.53	1.11	1.19	0.00

4.5.4 Conditions outside the training range

Figure 14 and Table 19 show five conditions beyond what the learned controllers saw in training. The PPO agent generalises in four of the five cases, with at least 98% utilisation and an RTT within 1.08 times the base, including a link 2.5 times faster and an RTT more than 2.5 times longer than any nominal training value. Under 8% random loss it falls to 82%, behind BBR and CUBIC+ML but far ahead of CUBIC. CUBIC+PPO does not generalise: it reaches only 30% utilisation under 8% loss and 31% with a 5-packet buffer. The hybrid was meant to be the design that holds up on unfamiliar links; in this implementation it is the controller that fails most often outside its training range.

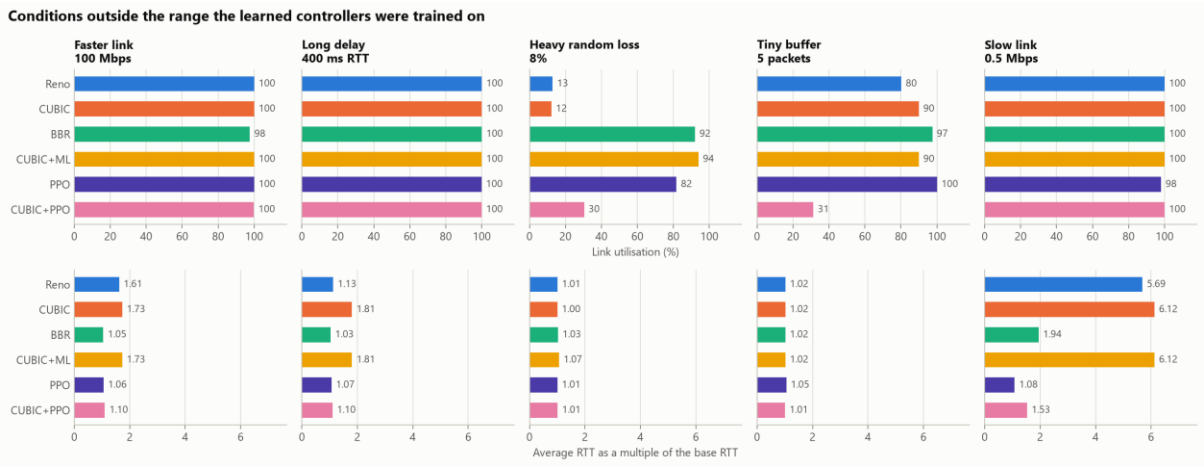


Figure 14: Utilisation (top) and delay ratio (bottom) under five conditions outside the training range

Table 19: Link utilisation (%) under conditions outside the training range

Controller	100 Mbps link	400 ms base RTT	8% random loss	5-packet queue	0.5 Mbps link
Reno	100.0	100.0	12.9	80.1	100.0
CUBIC	100.0	100.0	12.2	89.9	100.0
BBR	97.6	100.0	92.0	97.4	100.0
CUBIC+ML	100.0	100.0	93.9	89.9	100.0
PPO	100.0	99.8	81.7	100.0	98.1
CUBIC+PPO	99.9	99.9	30.3	31.3	100.0

4.5.5 Fairness

Figure 15 and Table 20 show how the controllers share a link. This is where the learned rate controller fails. Beside one CUBIC flow, PPO obtains 2.0% of the link and CUBIC+PPO 4.9%. CUBIC keeps the queue full; the agent reads the resulting delay as a signal to slow down, and CUBIC takes whatever it releases. This is the known weakness of delay-sensitive controllers, which Copa addresses with a separate competitive mode [10]. Two PPO flows also split a link unevenly, with a Jain index of 0.61. CUBIC+ML has neither problem: it takes 50.5% beside standard CUBIC, and two CUBIC+ML flows share equally.

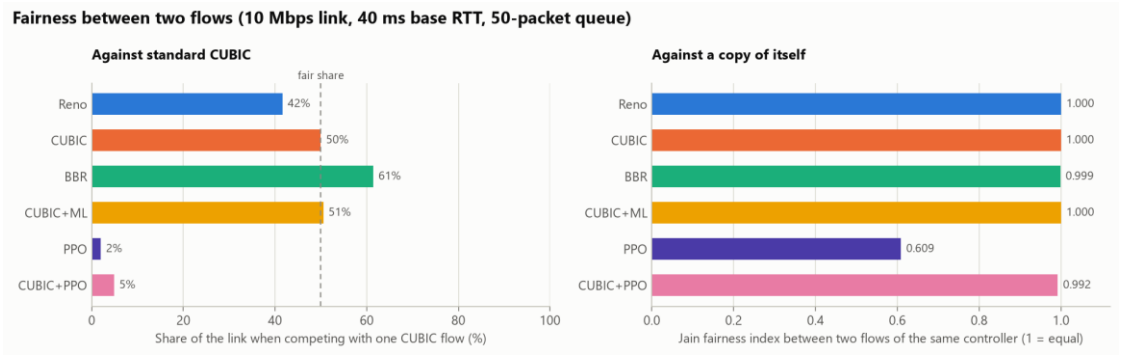


Figure 15: Share of a link beside one CUBIC flow (left) and Jain index between two flows of the same controller (right)

Table 20: Fairness between two flows on the standard link

Controller	Share beside CUBIC (%)	Share when started first (%)	Share when joined late (%)	Jain index, two of its own kind
Reno	41.6	44.6	38.6	1.000
CUBIC	50.0	48.9	51.1	1.000
BBR	61.4	61.4	61.4	0.999
CUBIC+ML	50.5	50.0	51.1	1.000
PPO	2.0	2.0	2.0	0.609
CUBIC+PPO	4.9	5.5	4.3	0.992

Stage 5 confirmed the classifier as a safe improvement and the agent as the best controller on a link of its own, but exposed the agent's main weakness: it starves beside a loss-based flow. Stage 6 addresses this, together with a second weakness that appears when measurement noise is added.

4.6 Stage 6: fixing the weaknesses in simulation

4.6.1 A competitive mode: PPO-Switch

Delay-based control has a known remedy for this failure: switch to a buffer-filling mode while another flow is filling the queue, as Copa does [10]. PPO-Switch applies the idea to the agent. It runs the learned policy, in the jitter-robust form described below, and hands control to CUBIC+ML on either of two signs that another flow holds the queue: a queueing delay above half the base RTT for six intervals in a row, or a rate that slides steadily to half over about 4 s while a moderate queue persists. The second sign covers a flow that joins a queue that is already full, whose minimum RTT is therefore inflated; a sudden delay spike is excluded, because it means that the capacity dropped. Every 4 s, doubling up to 16 s, it briefly lowers its window below the bandwidth-delay product, as BBR does to measure the minimum RTT; if the RTT then falls to the base RTT, the policy takes over again.

Table 21 shows the effect. Beside one CUBIC flow PPO-Switch obtains 47.4% of the link, against 2.0% for PPO, and it shares about equally with Reno and BBR as well. When it is alone it behaves exactly like PPO-J: it changed mode on none of the 30 single-flow test links and not on the link with changing capacity. Figure 16 shows the mechanism at work. While a CUBIC flow is present PPO obtains 0.2 Mbps; PPO-Switch changes mode within a second of CUBIC's arrival, obtains 4.5 of the 10 Mbps, and returns to low delay about 9 s after CUBIC leaves. Two limits remain: on a path with a 100 ms base RTT it obtains 24% within 60 s, where CUBIC converges slowly and even CUBIC+ML obtains 71% or 29% depending on the start order, and two PPO-Switch flows still share unevenly (Jain index 0.65).

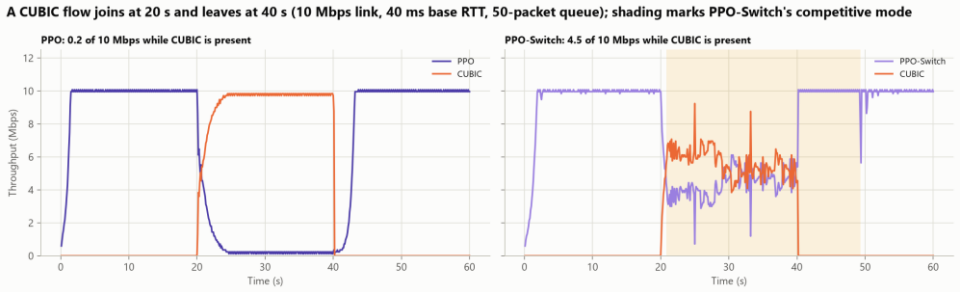


Figure 16: A CUBIC flow joins at 20 s and leaves at 40 s: PPO (left) and PPO-Switch, whose competitive mode is shaded (right)

Table 21: PPO-Switch compared with PPO, PPO-J and CUBIC+ML

Measure	PPO	PPO-J	PPO-Switch	CUBIC+ML
Share beside one CUBIC flow (%)	2.0	2.0	47.4	50.5
Share beside one Reno flow (%)	2.0	2.0	49.2	61.9
Share beside one BBR flow (%)	2.1	2.1	43.3	38.6
Utilisation on 30 unseen links, alone (%)	98.8	98.5	98.5	99.5
RTT / base RTT on those links	1.07	1.17	1.17	1.88
Utilisation with 4 ms RTT jitter (%)	2.8	99.2	99.2	100.0

4.6.2 Robustness to RTT jitter: PPO-J

On a real network every RTT sample carries a few milliseconds of random delay from wireless channel access and operating-system timers. The simulator was therefore given return-path jitter: an exponential extra delay, drawn every 2 ms and shared by the packets leaving the queue in that time, which never reorders ACKs and does not depend on the sending rate. Table 22 shows the utilisation of one flow as the mean jitter grows. The classical controllers and CUBIC+ML keep their throughput. PPO collapses, to 20.6% at 2 ms and 2.8% at 4 ms. Trained only on exact RTTs, it reads jitter as a growing queue and keeps slowing down. This matters in practice, because jitter on Wi-Fi regularly exceeds 2 ms.

PPO-J addresses it with two changes. It is trained with random jitter, of mean 0.3 to 10 ms, in 75% of its episodes, which is a form of domain randomisation, and its two latency signals are computed from the minimum RTT of each half interval instead of the mean. Jitter only ever adds delay, so it inflates the mean but hardly the minimum, whereas a real queue raises both; Copa filters its RTT samples in the same way [10]. PPO-J keeps 99.2% utilisation at 4 ms of jitter and 89.1% at 8 ms, and on the 30 unseen links without jitter it uses 98.5% of capacity, against 98.8% for PPO. The price is a slightly larger queue: an RTT of 1.17 times the base instead of 1.07. Jitter also exposes a weakness of BBR, whose bandwidth estimate is inflated by bursts of ACKs: its RTT rises from 1.05 to 2.0 to 2.6 times the base.

Table 22: Utilisation (%) of one flow on the standard link under random RTT jitter (mean of 2 runs)

Controller	0 ms	1 ms	2 ms	4 ms	8 ms
Reno	100.0	100.0	100.0	99.7	95.5
CUBIC	100.0	100.0	100.0	100.0	98.7
BBR	98.2	98.2	98.2	98.1	97.9
CUBIC+ML	100.0	100.0	100.0	100.0	98.8
PPO	100.0	99.4	20.6	2.8	2.5
CUBIC+PPO	100.0	98.4	73.0	22.9	9.7
PPO-J and PPO-Switch	99.9	99.8	99.7	99.2	89.1

After Stage 6 the agent, in simulation, shares a link fairly with classical flows and tolerates jitter. Whether this holds on a real network is the question of Stage 7.

4.7 Stage 7: test on a real Wi-Fi network

To check the results outside simulation, the same controller code was run over a real network. A Windows laptop sends UDP packets over Wi-Fi to a MacBook, which acknowledges every packet, so RTTs, jitter and losses come from a real wireless link and two real operating systems. To compare with the simulator on equal terms, the receiver places the standard bottleneck in front of itself: a 10 Mbps drop-tail queue of 50 packets, with extra delay for a 40 ms base RTT. Each controller ran alone for 30 s, again with 1% random loss added at the receiver, and beside one CUBIC flow for 60 s. The whole test was run twice, with nearly identical results.

Five recordings of the bare Wi-Fi path, 29 minutes in all, show what the controllers face (Figure 17). The link stalls about twice a second for roughly 75 ms; packets sent during a stall wait for its end and then arrive together. The median extra delay is only 3.7 ms, but 18% of packets are more than 20 ms late. With a 50-packet queue, the backlog of each stall floods the bottleneck when it ends, so on this path no sender can have both full throughput and low delay: a fixed rate of 6 Mbps keeps the RTT at 1.7 times the base, while 10 Mbps raises it to 2.9 times with 4.7% loss.

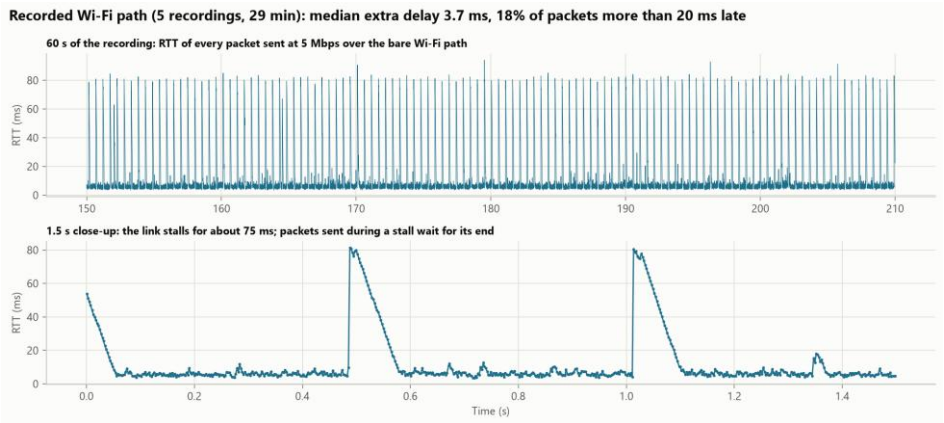


Figure 17: RTT of every packet on the recorded Wi-Fi path: 60 s (top) and a 1.5 s close-up of two stalls (bottom)

Table 23 compares each controller on the real path with the simulator, on the clean link and replaying the recorded delay. The replay applies the recorded delay to data on its way to the bottleneck and to ACKs on their way back, each until the stall ends, and uses only the last 30% of every recording. It predicts each controller's real throughput within 1.3 Mbps, including the 33% loss of BBR, whose bandwidth estimate is inflated by bursts of ACKs (25% predicted). The classical controllers and CUBIC+ML keep 7.6 to 9.4 Mbps. With 1% random loss CUBIC+ML keeps 8.7 Mbps and CUBIC 2.9, so the benefit of the loss classifier carries over to the real network, and PPO-Switch takes 45% of the link beside CUBIC in both runs, so the competitive mode does too.

Table 23: Each controller alone: simulator, replay of the recorded Wi-Fi, and the real Wi-Fi path (second run)

Controller	Simulator, clean link (Mbps)	Replay of recorded Wi-Fi (Mbps)	Real Wi-Fi (Mbps)	Real RTT (ms)
Reno	10.0	8.9	7.6	72
CUBIC	10.0	9.1	8.6	70
BBR	9.9	9.3	9.4 (33% loss)	105
CUBIC+ML	10.0	9.2	8.8	79
PPO	10.0	0.2	0.3	51
CUBIC+PPO	9.9	0.8	1.2	48
PPO-J	10.0	3.9	3.3	54
PPO-W	9.8	6.0	5.8	63
PPO-Switch	10.0	3.9	3.6	55

The learned rate controllers do not carry over as well: PPO collapses to 0.3 Mbps and PPO-J reaches 3.3, because each stall looks like congestion to a delay-based agent. Stage 7 therefore showed which results transfer to a real network, and that the remaining gap lies in the conditions the agent was trained on.

4.8 Stage 8: learning from the real network (PPO-W)

To close the gap, PPO-W was trained like PPO-J but with the first 70% of each recording replayed in half of its episodes. A first version learned to hold back, because its reward also penalised stall delay that no sender can remove; scoring delay by the queue at the bottleneck instead, which only the reward sees, gave the final policy. On the held-out replay it reaches 60% utilisation against 39% for PPO-J, close to the best that any fixed rate achieves on this path for its delay. On the real Wi-Fi it delivers 5.8 Mbps, as the replay predicted (6.0), with an RTT of 63 ms against 70 ms for CUBIC, which still delivers more (8.6 Mbps) by accepting a full queue. On the 30 clean unseen links PPO-W uses 96.6% of capacity at 1.27 times the base RTT. Without a competitive mode it takes 2.4% beside CUBIC.

4.9 What each stage achieved

Table 24 lists what each stage did, what it found and what it led to.

Table 24: What each stage did, found and led to

Stage	What we did	What we found	What it led to
1	Simulator; Reno, CUBIC and BBR	Reno and CUBIC double the RTT; CUBIC falls to 3.7 Mbps at 1% loss; Reno matches the Mathis model	The two weaknesses to remove
2	XGBoost loss classifier inside CUBIC	91.4% accuracy; 9.6 Mbps at 5% loss (CUBIC 1.6); fair to CUBIC	Random-loss weakness removed
3	LSTM forecast of the RTT 0.5 s ahead	12% lower error than repeating the last value	A modest, stand-alone result
4	PPO agent that sets the sending rate	98.8% of capacity at 1.07 times the base RTT	A candidate fix for full queues
5	Six controllers on unseen, lossy, changing and extreme links	Agent best when alone, but 2% of a link beside CUBIC	Fix the agent (Stage 6)
6	PPO-Switch; PPO-J trained with jitter	47% beside CUBIC; 99% at 4 ms of jitter (PPO 3%)	Test on a real network
7	Every controller over Wi-Fi between two laptops	Classifier and competitive mode carry over; agent 0.3 to 3.3 Mbps; Wi-Fi stalls 75 ms twice a second	Train on the real network
8	29 min of Wi-Fi recorded; PPO-W trained on it	5.8 Mbps on real Wi-Fi (6.0 predicted); 63 ms RTT against 70 ms for CUBIC	The conclusions of Chapter 5

5. Conclusion

5.1 What we did and what we achieved

The project set out to test whether learned components can remove the two weaknesses of rule-based congestion control, and progressed in eight stages (Figure 1, Table 24). A loss classifier inside CUBIC removed the collapse under random loss without making CUBIC unfair, and a PPO agent removed the full queue for a flow alone at the bottleneck. The agent's two weaknesses, starvation beside CUBIC and sensitivity to jitter, were fixed in simulation; on a real Wi-Fi network the classifier and the competitive mode held up, and PPO-W, trained on recordings of that network, raised the agent from 0.3 to 5.8 Mbps.

5.2 What we take from this project

1. A small learned component inside a proven algorithm is the most reliable gain: CUBIC+ML changed only how losses are read, stayed fair, and kept its benefit on the real network.
2. Reinforcement learning beats hand-written rules on a link of its own, but this does not transfer to sharing; more training did not fix starvation beside CUBIC, an explicit competitive mode did.
3. A learned controller is only as good as its training conditions: trained on exact RTTs, the agent failed with 2 ms of jitter and on real Wi-Fi.
4. Measuring the real network and replaying it works: 29 minutes of recordings predicted every controller's real throughput within 1.3 Mbps, and training on them raised the agent from 0.3 to 5.8 Mbps.
5. What the reward counts matters as much as the data: counting only the queue the agent builds, not delay it cannot control, raised its utilisation on the replay from 42% to 60%.
6. A simulated result is a hypothesis until it is tested on a real network: the simulator ranked the agent first, but on real Wi-Fi CUBIC delivered more.

The answer to the question of Chapter 1 is therefore yes, with qualifications. Learned components removed both weaknesses in simulation, and the loss classifier removed one of them on a real network too. Learned rate control needs real measurements, a carefully chosen reward and an explicit mechanism for sharing; it is a promising direction rather than a ready replacement for CUBIC.

5.3 Limitations and future work

The real-network test covers one Wi-Fi path, in user-space UDP, and PPO-W is trained on that path; the simulator has no retransmission timeouts and a single bottleneck. Two learned flows still share a link unevenly. Future work should train agents against each other, as Astraea does, and test on more networks and across the Internet.

References

[1] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. Pearson, 2021.

[2] V. Jacobson, "Congestion avoidance and control," in Proc. ACM SIGCOMM, 1988, pp. 314–329.

[3] D.-M. Chiu and R. Jain, "Analysis of the increase and decrease algorithms for congestion avoidance in computer networks," Computer Networks and ISDN Systems, vol. 17, no. 1, pp. 1–14, 1989.

[4] M. Allman, V. Paxson, and E. Blanton, "TCP congestion control," RFC 5681, Internet Engineering Task Force, Sep. 2009.

[5] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," ACM SIGOPS Operating Systems Review, vol. 42, no. 5, pp. 64–74, 2008.

[6] L. Xu, S. Ha, I. Rhee, V. Goel, and L. Eggert, Ed., "CUBIC for fast and long-distance networks," RFC 9438, Internet Engineering Task Force, Aug. 2023.

[7] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," ACM Queue, vol. 14, no. 5, pp. 20–53, 2016.

[8] H. Jiang, Q. Li, Y. Jiang, G. Shen, R. Sinnott, C. Tian, and M. Xu, "When machine learning meets congestion control: A survey and comparison," Computer Networks, vol. 192, 2021.

[9] L. S. Brakmo and L. L. Peterson, "TCP Vegas: End to end congestion avoidance on a global Internet," IEEE Journal on Selected Areas in Communications, vol. 13, no. 8, pp. 1465–1480, 1995.

[10] V. Arun and H. Balakrishnan, "Copa: Practical delay-based congestion control for the Internet," in Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2018.

[11] S. Floyd and V. Jacobson, "Random early detection gateways for congestion avoidance," IEEE/ACM Transactions on Networking, vol. 1, no. 4, pp. 397–413, 1993.

[12] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan, "Data center TCP (DCTCP)," in Proc. ACM SIGCOMM, 2010, pp. 63–74.

[13] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," ACM SIGCOMM Computer Communication Review, vol. 27, no. 3, Jul. 1997.

[14] J. Padhye, V. Firoiu, D. Towsley, and J. Kurose, "Modeling TCP throughput: A simple model and its empirical validation," in Proc. ACM SIGCOMM, 1998, pp. 303–314.

[15] R. Jain, D.-M. Chiu, and W. R. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corporation, Research Report DEC-TR-301, Sep. 1984.

[16] S. Biaz and N. H. Vaidya, "Discriminating congestion losses from wireless losses using inter-arrival times at the receiver," in Proc. IEEE Symposium on Application-Specific Systems and Software Engineering and Technology (ASSET), 1999, pp. 10–17.

[17] S. Mascolo, C. Casetti, M. Gerla, M. Y. Sanadidi, and R. Wang, "TCP Westwood: Bandwidth estimation for enhanced transport over wireless links," in Proc. ACM MobiCom, 2001, pp. 287–297.

[18] C. P. Fu and S. C. Liew, "TCP Veno: TCP enhancement for transmission over wireless access networks," IEEE Journal on Selected Areas in Communications, vol. 21, no. 2, Feb. 2003.

[19] K. Winstein and H. Balakrishnan, "TCP ex Machina: Computer-generated congestion control," in Proc. ACM SIGCOMM, 2013.

[20] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "PCC: Re-architecting congestion control for consistent high performance," in Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2015, pp. 395–408.

[21] M. Dong, T. Meng, D. Zarchy, E. Arslan, Y. Gilad, B. Godfrey, and M. Schapira, "PCC Vivace: Online-learning congestion control," in Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2018, pp. 343–356.

[22] W. Li, F. Zhou, K. R. Chowdhury, and W. Meleis, "QTCP: Adaptive congestion control with reinforcement learning," IEEE Transactions on Network Science and Engineering, vol. 6, no. 3, pp. 445–458, 2019.

[23] F. Y. Yan, J. Ma, G. D. Hill, D. Raghavan, R. S. Wahby, P. Levis, and K. Winstein, "Pantheon: The training ground for Internet congestion-control research," in Proc. USENIX Annual Technical Conference (ATC), 2018.

[24] N. Jay, N. Rotman, B. Godfrey, M. Schapira, and A. Tamar, "A deep reinforcement learning perspective on Internet congestion control," in Proc. International Conference on Machine Learning (ICML), PMLR vol. 97, 2019, pp. 3050–3059.

[25] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Classic meets modern: A pragmatic learning-based congestion control for the Internet," in Proc. ACM SIGCOMM, 2020.

[26] C.-Y. Yen, S. Abbasloo, and H. J. Chao, "Computers can learn from the heuristic designs and master Internet congestion control," in Proc. ACM SIGCOMM, 2023.

[27] X. Liao, H. Tian, C. Zeng, X. Wan, and K. Chen, "Astraea: Towards fair and efficient learning-based congestion control," in Proc. European Conference on Computer Systems (EuroSys), 2024, pp. 99–114.

[28] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[29] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[30] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.

[32] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018.

[33] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," Journal of Machine Learning Research, vol. 22, no. 268, pp. 1–8, 2021.

[34] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019.

[35] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.